

# END-OF-INTERNSHIP PRESENTATION

**Muthukumaran Yogeeswaran**

Advanced Robotics Applications

Supervisor: Yan Shijun

Date: 26/07/2024



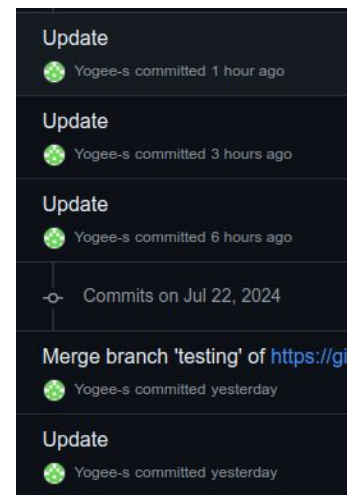
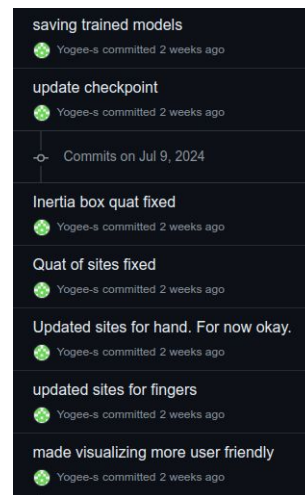
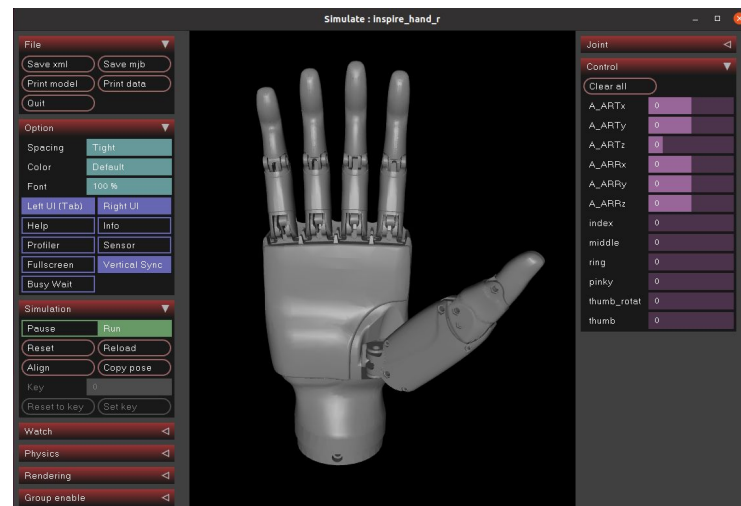
# Content

|                                        |                |
|----------------------------------------|----------------|
| <b>Skills learnt during internship</b> | <b>Page 3</b>  |
| <b>Project sharing</b>                 | <b>Page 4</b>  |
| <b>Appreciation to ARTC/ team</b>      | <b>Page 17</b> |
| <b>Feedback to ARTC/ team</b>          | <b>Page 18</b> |



# Skills learnt during internship

- Reinforcement Learning with demonstrations
- Mujoco Simulation
- Managing Python environments using Conda
- Debugging with VScode
- Version control in Git with branching

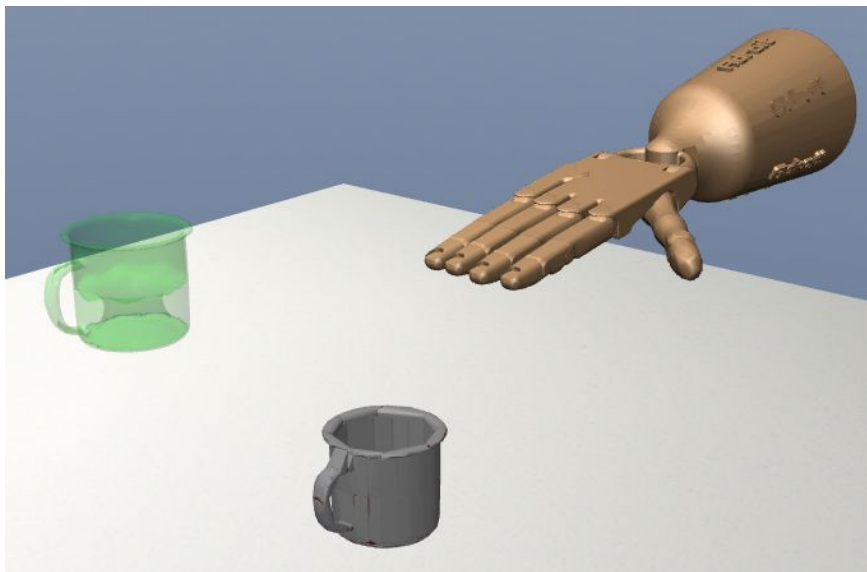


# Project Sharing

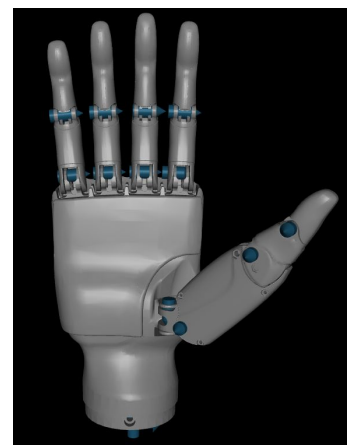




## Main Goal (Adapt to Inspire hand)



Trained using a **Demo Augmented Policy Gradients (DAPG)** algorithm





# Mujoco (URDF → XML)

```
<robot
  name="inspire_hand_r">

  <link name="world"/>
  <joint name="fixed" type="fixed">
    <origin
      xyz="0.5 0.5 0.5"
      rpy="-1.5707963 0 0" />
    <parent link="world"/>
    <child link="R_hand_base_link"/>
  </joint>

  <link
    name="R_hand_base_link">
    <inertial>
      <origin
        xyz="-0.076223 -0.01701 0.24616"
        rpy="0 0 0" />
      <mass
        value="0.14143" />
      <inertia
        ixx="7.6663E-05"
        ixy="1.6551E-06"
        ixz="1.7709E-06"
        iyy="8.3832E-05"
```

```
<mujoco model="inspire_hand_r">
  <compiler angle="radian" />
  <option integrator="RK4" timestep="0.001" />
  <size njmax="2000" nconmax="500" nstack="600000"/>

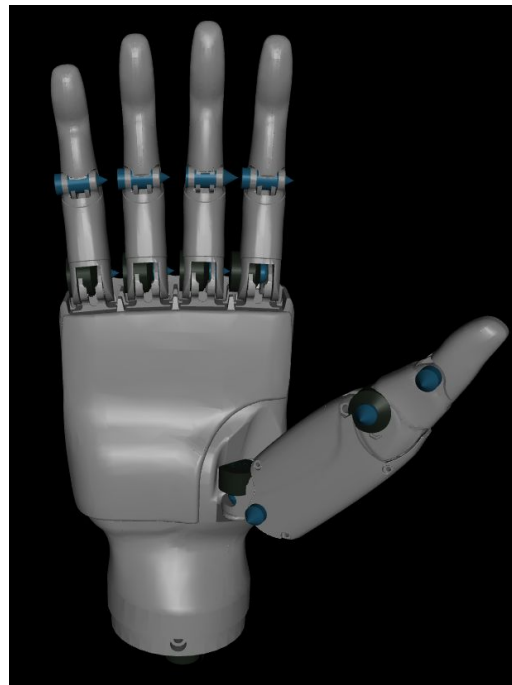
  <asset>
    <mesh name="R_hand_base_link" file="../meshes/R_hand_base_link.STL" />
    <mesh name="R_thumb_proximal_base" file="../meshes/R_thumb_proximal.STL" />
    <mesh name="R_thumb_proximal" file="../meshes/R_thumb_proximal.STL" />
    <mesh name="R_thumb_intermediate" file="../meshes/R_thumb_intermediate.STL" />
    <mesh name="R_thumb_distal" file="../meshes/R_thumb_distal.STL" />
    <mesh name="R_index_proximal" file="../meshes/R_index_proximal.STL" />
    <mesh name="R_index_distal" file="../meshes/R_index_distal.STL" />
    <mesh name="R_middle_proximal" file="../meshes/R_middle_proximal.STL" />
    <mesh name="R_middle_distal" file="../meshes/R_middle_distal.STL" />
    <mesh name="R_ring_proximal" file="../meshes/R_ring_proximal.STL" />
    <mesh name="R_ring_distal" file="../meshes/R_ring_distal.STL" />
    <mesh name="R_pinky_proximal" file="../meshes/R_pinky_proximal.STL" />
    <mesh name="R_pinky_distal" file="../meshes/R_pinky_distal.STL" />
  </asset>

  <worldbody>
    <body name="hand" pos="-0.42 -0.75 -0.6">
      <inertial pos="0.42 0.5 0.75" quat="0 0 0.707 0.707" mass="0.14143" />
      <joint name="ABTx" pos="0.42 0.4 0.75" axis="-1 0 0" type="revolute" />
```



# Mujoco XML

- Equality constraints to group joints for actuators
- Free joints to move hand (6 DOF → 12 DOF)
- Adding collision properties
- Damping
- Actuator/Joint ranges



```
<worldbody>
  <body name="hand" pos="-0.42 -0.75 -0.6">
    <inertial pos="0.42 0.5 0.75" quat="0 0 0.707 0.707" mass="0.3" diaginertia="0.01 0.01 0.0075"/>
    <joint name="ARTx" pos="0.42 0.4 0.75" axis="-1 0 0" type="slide" range="-0.75 0.75"
      damping="20" limited="true" margin="0.01"
      armature="0.001" frictionloss="0.001"/>
```



# Data fitting

Overview of data loaded from the last batch in the pickle file:

Variable: model\_data, Type: <class 'list'>, List of length 1

Variable: observations, Type: <class 'numpy.ndarray'>, Numpy array of shape (521, 39), size: 20319

Variable: rewards, Type: <class 'numpy.ndarray'>, Numpy array of shape (521,), size: 521

Variable: actions, Type: <class 'numpy.ndarray'>, Numpy array of shape (521, 30), size: 15630

Variable: sim\_data, Type: <class 'list'>, List of length 521

87







### Observations (39)

**30 Joints**

+

**9 pos (30 to 38)**

(distance in x,y,z)

Hand to Object

Hand to target

Object to target



### Observations (21)

**12 Joints**

+

**9 pos**

(distance in x,y,z)

Hand to Object

Hand to target

Object to target

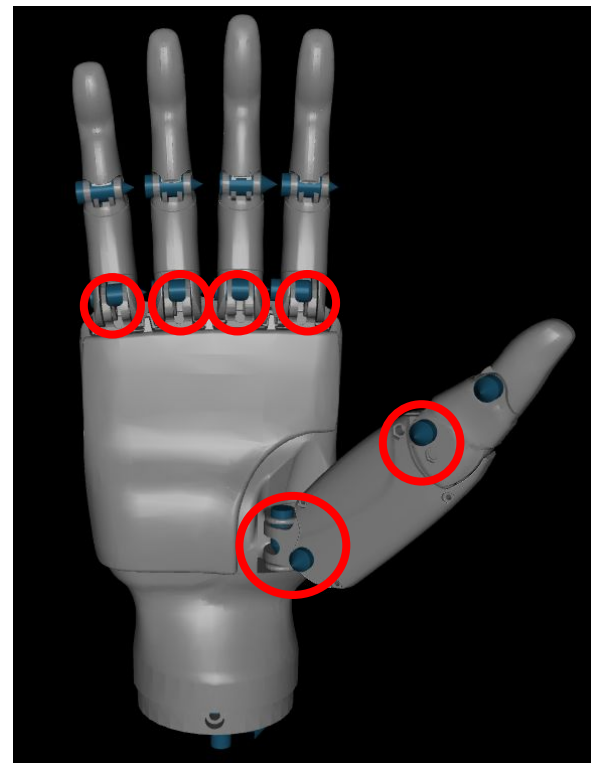
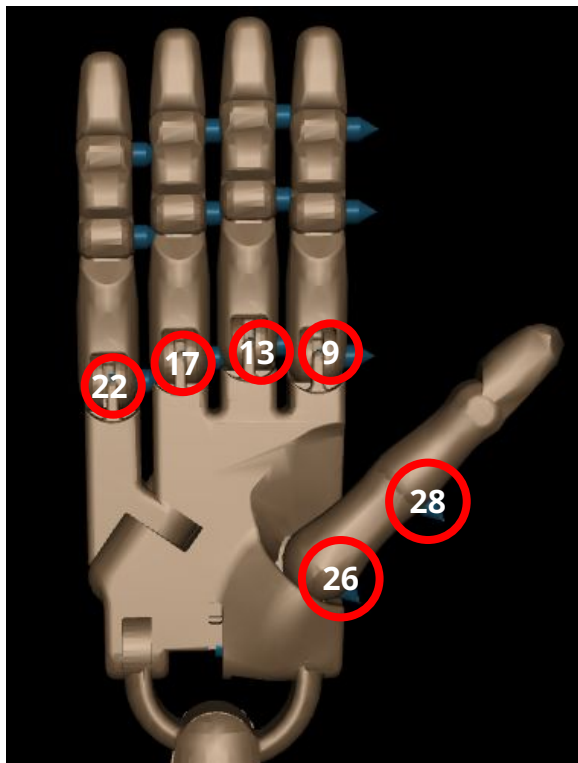
```
def train from paths(self, paths):
    # Finger with 12DOF
    obs_indexes = [0, 1, 2, 3, 4, 5, 9, 13, 17, 22, 26, 28, 30, 31, 32, 33, 34, 35, 36, 37, 38]
    act_indexes = [0, 1, 2, 3, 4, 5, 9, 13, 17, 22, 26, 28]
    # Concatenate from all the trajectories
    observations = np.concatenate([path["observations"] for path in paths])
    actions = np.concatenate([path["actions"] for path in paths])
    advantages = np.concatenate([path["advantages"] for path in paths])
    advantages = (advantages - np.mean(advantages)) / (np.std(advantages) + 1e-6)

    if self.demo_paths is not None and self.lam_0 > 0.0:
        demo_obs = np.concatenate([path["observations"][:, obs_indexes] for path in self.demo_paths])
        demo_act = np.concatenate([path["actions"][:, act_indexes] for path in self.demo_paths])
        demo_adv = self.lam_0 * (self.lam_1 ** self.iter_count) * np.ones(demo_obs.shape[0])
        self.iter_count += 1
        # concatenate all
        all_obs = np.concatenate([observations, demo_obs])
        all_act = np.concatenate([actions, demo_act])
        all_adv = cfg.DAPG_ADV_W * np.concatenate([advantages / (np.std(advantages) + 1e-8), demo_adv])
```



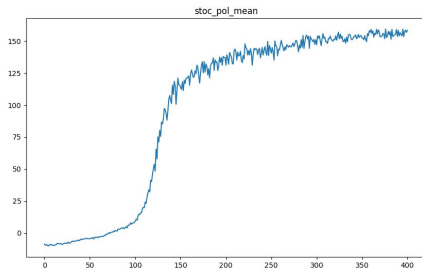
# Mapping joints

```
def train_from_paths(self, paths):  
    # Finger with 12DOF  
    obs_indexes = [0, 1, 2, 3, 4, 5, 9, 13, 17, 22, 26, 28, 30, 31, 32, 33, 34, 35, 36, 37, 38]  
    act_indexes = [0, 1, 2, 3, 4, 5, 9, 13, 17, 22, 26, 28]  
    # Concatenate from all the trajectories  
    observations = np.concatenate([path["observations"] for path in paths])
```



# Challenges faced

- Model reaching **local maximum**
  - Found the wrong way to do something



- **Limited time** to try out new things
  - Training takes time to produce results
- Training is **not consistent**
  - Every instance of training could be different





# Parameters configured

```
DEMO_FILE: ../demonstrations/relocate-mug.pkl
ENV_NAME: relocate-mug-0.8
JOB_DIR: ./training_log
JOB_NAME: trpo
POLICY_INIT_LOG_STD: 0.0
POLICY_WS:
  - 32
  - 32
RNG_SEED: 200
NUM_CPU: 10
```



```
#Parameters configured
```

```
#####
```

```
NUM_ITER: 2000
```

```
# BC_INIT: true
```

```
USE_DAPG: true
```

```
DAPG_LAM0: 0.8
```

Demo Coefficient

```
DAPG_LAM1: 0.99
```

Decay Coefficient

```
SAVE_FREQ: 50
```

```
# CHECKPOINT_POLICY: "/home/xiaoyu/Desktop/yogee/dexmv-sim/trained_models_mug/policy_1000.pickle"
```

```
# CHECKPOINT_BASELINE: "/home/xiaoyu/Desktop/yogee/dexmv-sim/trained_models_mug/baseline_1000.pickle"
```

```
#####
```



# Reward Structure

```
def reward(self, action):
    obj_pos = self.data.body_xpos[self.obj_bid].ravel()
    palm_pos = self.data.site_xpos[self.S_grasp_sid].ravel()
    target_pos = self.data.body_xpos[self.target_object_bid].ravel()
    is_contact = self.check_contact(self.body_geom_names, self.robot_geom_names)

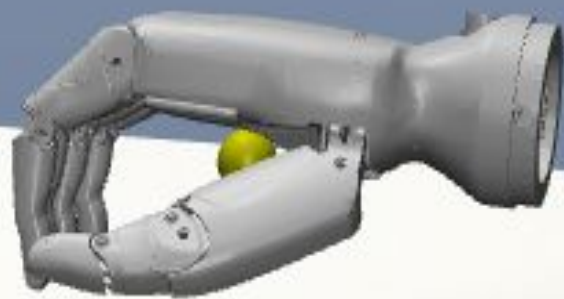
    reward = -0.1 * np.linalg.norm(palm_pos - obj_pos) # take hand to object

    if is_contact:
        reward += 0.1
        lift = max(min(obj_pos[2], target_pos[2]) - YCB_SIZE[self.object_name][2] / 2.0, 0)
        reward += 50 * lift
        if lift > 0.015: # if object off the table
            obj_target_distance = np.linalg.norm(obj_pos - target_pos)
            reward += 2.0 # bonus for lifting the object
            reward += -0.5 * np.linalg.norm(palm_pos - target_pos) # make hand go to target
            reward += -1.5 * obj_target_distance # make object go to target

            if obj_target_distance < 0.1:
                reward += 1 / (obj_target_distance + 0.01)

    return reward
```





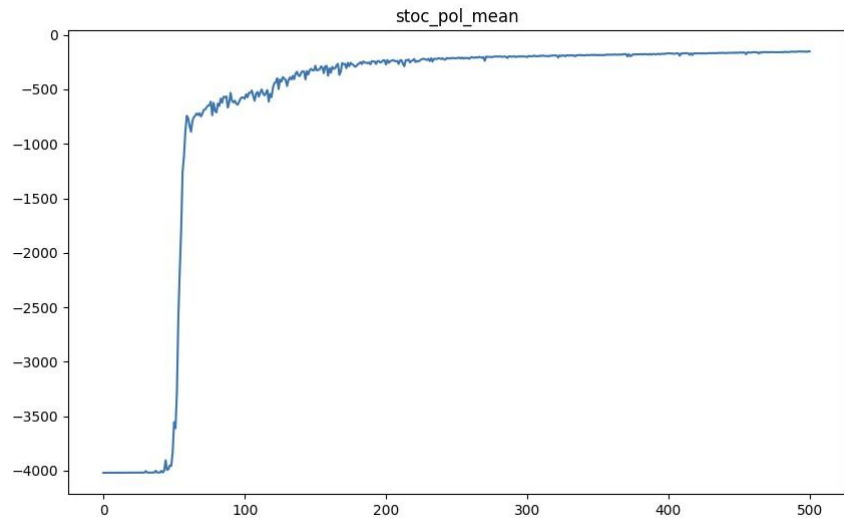




# Modified Reward Structure

```
# Calculate the vertical angle of the object
obj_quat = self.data.body_xquat[self.obj_bid].ravel()
obj_rot = transforms3d.quaternions.quat2mat(obj_quat)
vertical_angle = np.arccos(obj_rot[2, 1])

# Penalty for not being upright
upright_penalty = 1 - np.exp(-vertical_angle**2 * 5)
reward -= upright_penalty
```



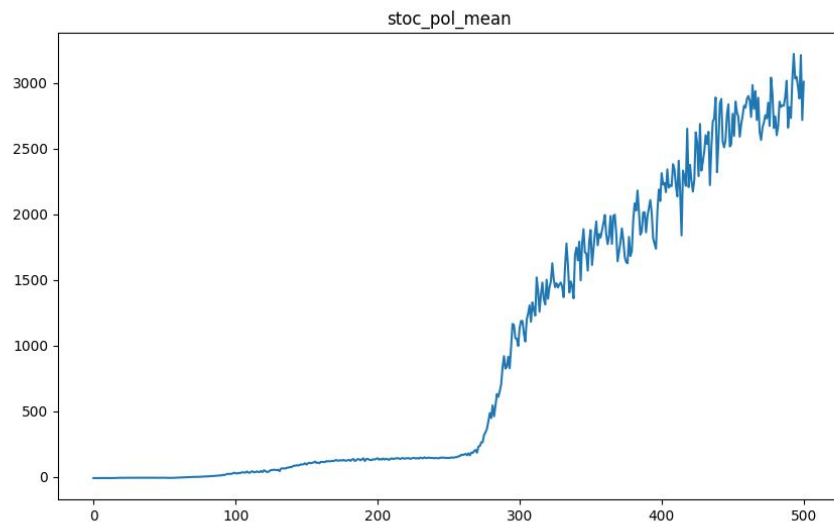
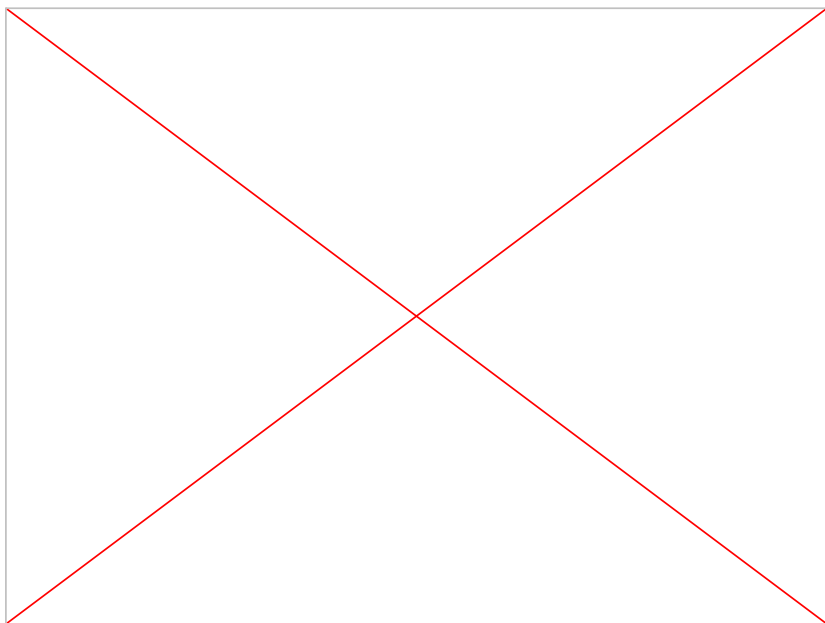
```
# Penalty for not being upright
upright_penalty = 1 - np.exp(-vertical_angle**2 * 5) # Stronger
reward -= upright_penalty
```

```
# Check if the object is toppled over
toppled_threshold = np.deg2rad(80) # Threshold for considering the
if vertical_angle > toppled_threshold:
    reward -= 10 # Large penalty if the object is toppled over

return reward
```

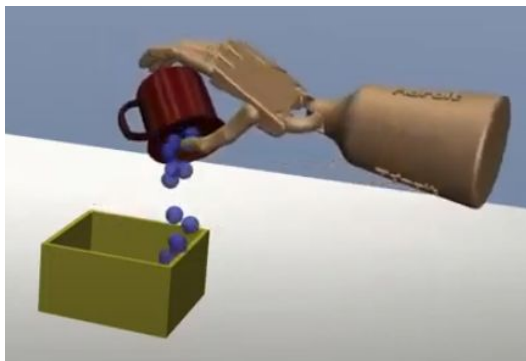


## Decent results (Mustard bottle → Mug)



## Moving forward

- **Integrate** trained model with **UR robot**
- Train model to **generalise** to different objects
  - Currently each object has its own model
- Train model to do more **complex tasks**





## Appreciation to ARTC/team

Yan Shijun (Supervisor)

Guan Mingyang (Internship POC)

Rukshan Hettiarachchi (Senior Research Engineer)

Sanjeev Raveendran (Intern)

Deng Haoyuan (Intern)



## Feedback to ARTC/team

- Diverse and Challenging Learning Opportunities
- Opportunity to experience staff meeting
- Access to Better Hardware or explore AWS to improve computational power
- Initiation programme for interns to get to know each other



CREATING GROWTH, ENHANCING LIVES



# THANK YOU

---

[www.a-star.edu.sg](http://www.a-star.edu.sg)